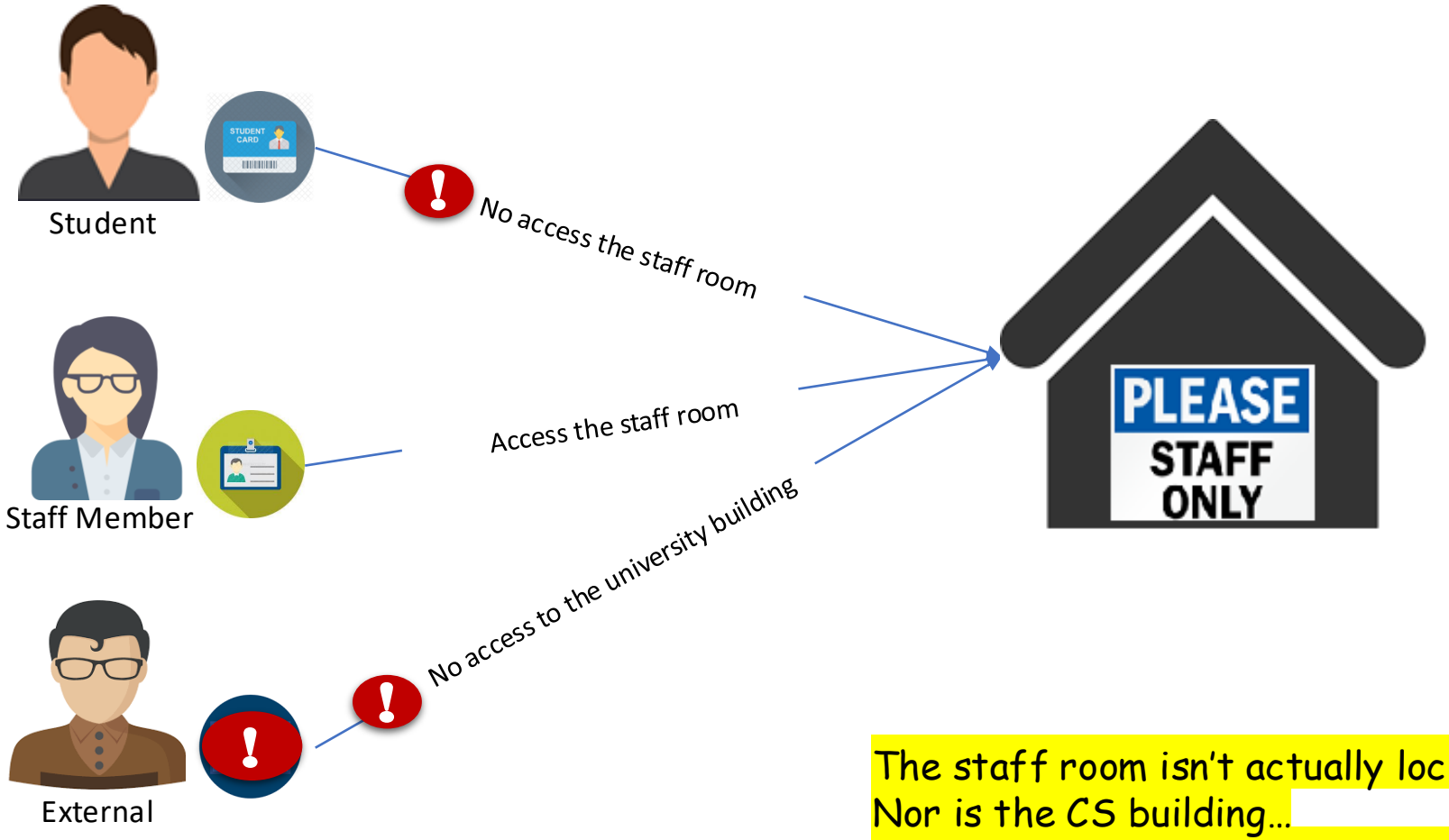
University of BRISTOL
School of Computer Science

Access Control

Hastily covered by Gretch Hallett
Email: gretch.hallett@bristol.ac.uk

~~Dr. Sana Belguith~~

Access Control Example



The staff room isn't actually locked…
Nor is the CS building…
And even if it were, we probably don't want to enforce it

Access Control Example

Identification



Student



Staff Member



External

Present Identity

Authentication



Student



Staff Member



External

Present Photo ID

Enforcement Mechanism

It's always a fun game to see
If you can convince this one to
Let you in anyway… I'm staff and
If you don't let me in you're gonna
Have to tell the students the lecture's
Cancelled works well…



Check ID by Security Agent

Authorisation



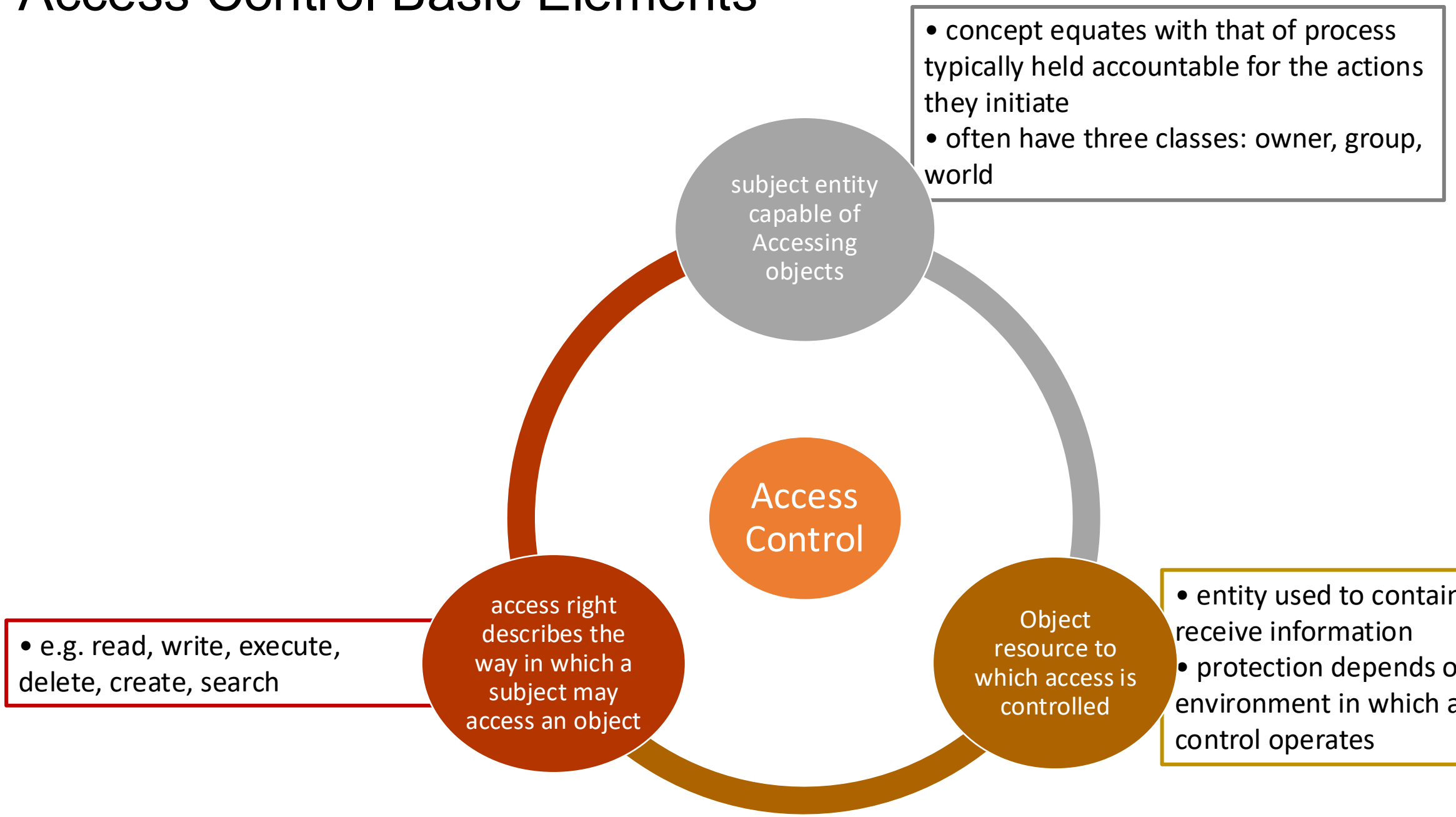
Staff Member Access

What is Access Control



- The ability to allow only authorized users, programs or processes system or resource access
- The granting or denying, according to a particular security model, of certain permissions to access a resource
- An entire set of procedures performed by hardware, software and administrators, to monitor access, identify users requesting access, record access attempts, and grant or deny access based on pre-established rules.

Access Control Basic Elements



Access Control < Authorization Logic

That approach to access control is the traditional "high level" one… but access control has been around FOREVER.

The earliest academic approaches were Blaze et al.'s *PolicyMaker* and *KeyNote* which acted as Trust Management systems…

What we want is a signed attestation from a trusted authority (a signed permission slip) from *someone* to say this action is okay…

Here the *authority* is the signer, the *principles* are any people involved and the *subject* is whatever is on the permission slip. The *policy* is the all the rules stapled together…

$\Gamma \models$ *authority says subject may X*

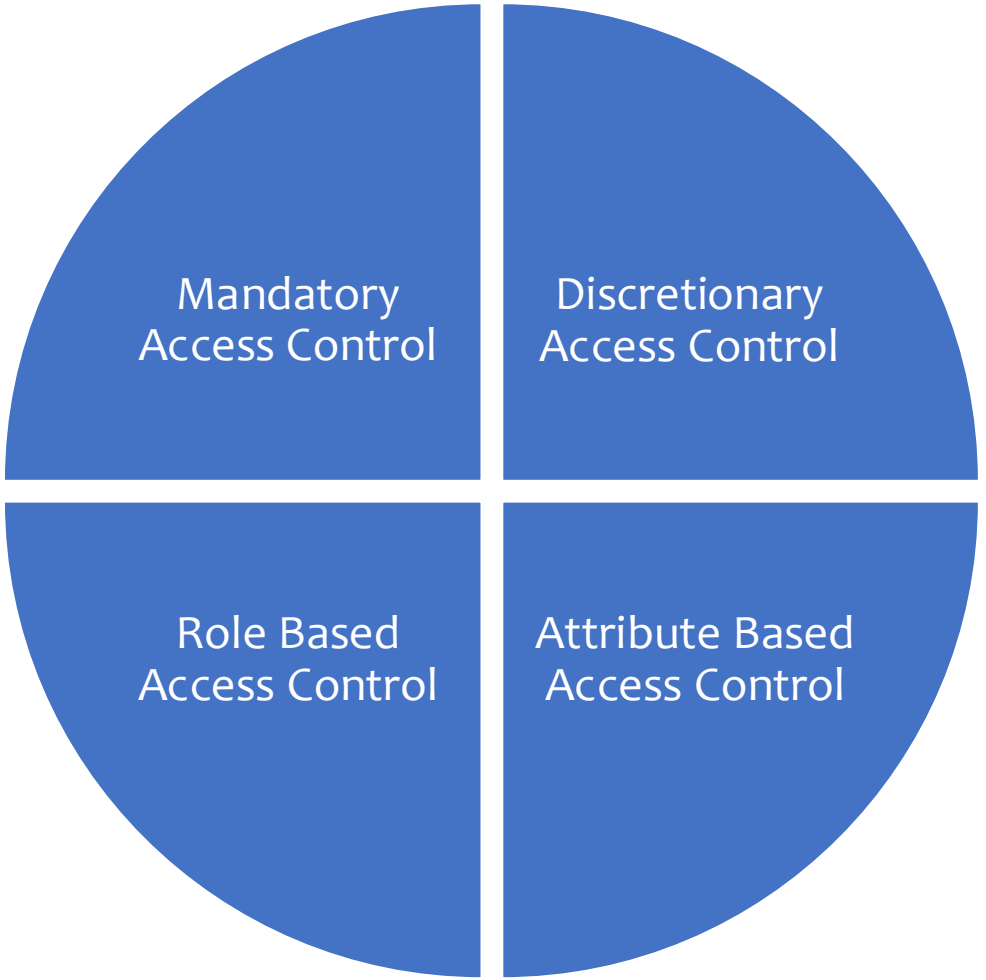
Access Control: Domains and Applications

- Social Networks
- Web Browsers
- Operating Systems
- Firewalls

Okay so these are some places you may have a Heading in the settings called "Access Control" But in reality this is being enforced every where Just because any authorization decision *will* have A policy associated with it… even if that policy is Just "yes".

Formally: $\Gamma \models \top$

Access Control Methods



Again, this is an oversimplification.

Various styles of access control policy exist, and any system may implement as many of them as it likes when describing what is permitted.

These terms are just used to describe broad strategies…

Ultimately its all just a question of style

Multiple Access Control Policies. DAC, MAC, RBAC are not mutually exclusive.
A system may implement two or even three of these policies for some or all types of access.

Discretionary Access Control (DAC)

Oh no…

I'm going to get all grumpy here…

- scheme in which an entity may enable another entity to access some resource
- often provided using an access matrix
  - one dimension consists of identified subjects that may attempt data access to the resources
  - the other dimension lists the objects that may be accessed
- each entry in the matrix indicates the access rights of a particular subject for a particular object
- rule enforcement may be waived or modified by some users.

Discretionary access control is where the subject itself carries the authorization policy, and it is capable of passing that policy to another authority for enforcement (it's at the subject's discretion)

In the case of the UNIX DAC, this is at the file owner's discretion.

First formalized by the US Department of Defence in the Trusted Systems Evaluation Criteria (TSEC 🌈)

This is about *you* controlling *who can do what* with *your stuff*

Access Control Matrix

- ACM is a matrix of all principals and objects.
- The matrix entries describe the permissions for Subjects, Objects, and operations
- Can determine
  - Who can access an object
  - What objects can be accessed by a subject
  - What operations a subject can perform on an object

- Problem: maintaining such a matrix can be difficult.
- If the matrix is corrupted then all controls is lost.

Ehhh… no harder or more corruptible than storing any other bit of information…

Access Control Matrix

- Suppose the private key file for J is object O1: Only J can read the key
- Suppose the public key file for J is object O2: All can read, only J can modify
- Suppose all can read and write from object O3

- What's the access matrix?

	O1	O2	O3
J	R	R, W	R, W
S2	N	R	R, W
S3	N	R	R, W

Least Privilege

Least privilege is that the security policy ought to allow a principle the minimum number of actions to complete their work.

- The Principle of Least Privilege states that a subject should be only given those privileges needed for it to complete its task.
- If a subject does not need an access right, the subject should not have that right.

	O1	O2	O3
J	R	R, W	N
S2	N	R	R, W
S3	N	R	R, W

- Limit permissions to those required and no more
- Consider three processes for user J
- Restrict privilege of the process O3 to prevent leaks

The umask on a Linux system by default is 022 (group and other may not write to a new file)… is this failing to meet least privilege?

Is this a vulnerability or a permissive default? Don't get too het up about *least privilege*…

Access Control Lists (ACLs)

This is just storing a matrix in adjacency list form…

It's still a massive matrix, just stored differently.

- We do not want to store one massive matrix.
- Instead we can store each column of the matrix with the object it refers to.
- For example:

  (O2,(J,RW), (S2,R), (S3,R))

	O1	O2	O3
J	R	R, W	R, W
S2	N	R	R, W
S3	N	R	R, W

Traditional UNIX File Access Control

I'm going to assume you all actually know this, because you've been using Linux for years at this point?

Yell if I need to go slowly!

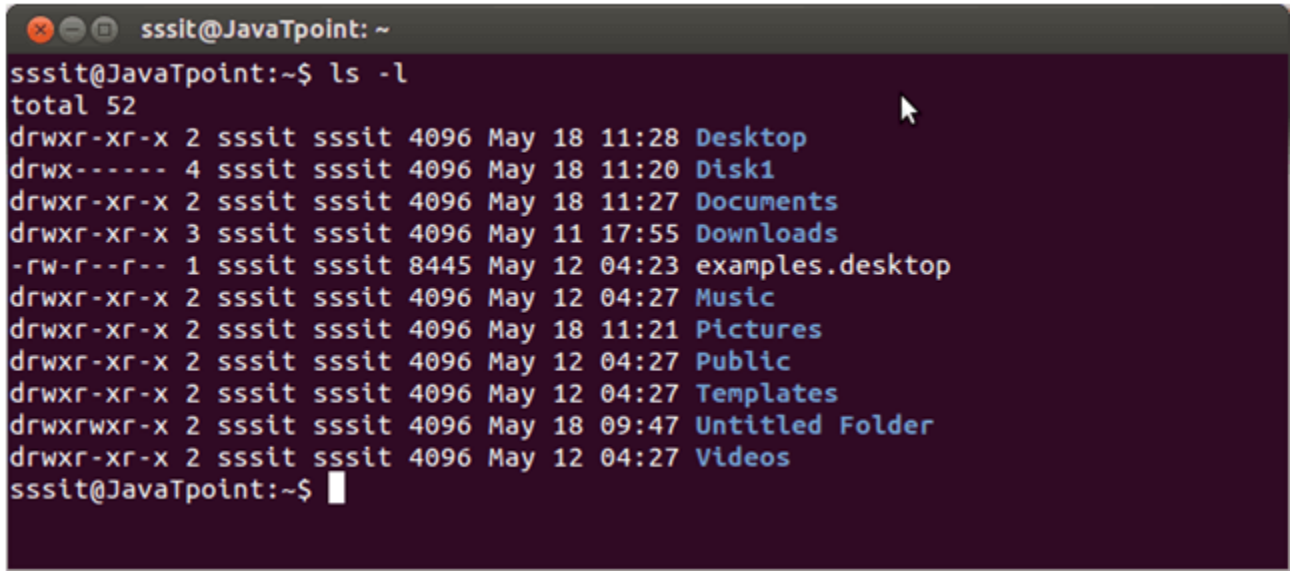
- "set user ID"(SetUID)
- "set group ID"(SetGID)
  - system temporarily uses rights of the file owner / group in addition to the real user's rights when making access control decisions
  - enables privileged programs to access files / resources not generally accessible
- superuser
  - is exempt from usual access control restrictions
  - has system-wide access

Access Control Lists (ACLs) in UNIX

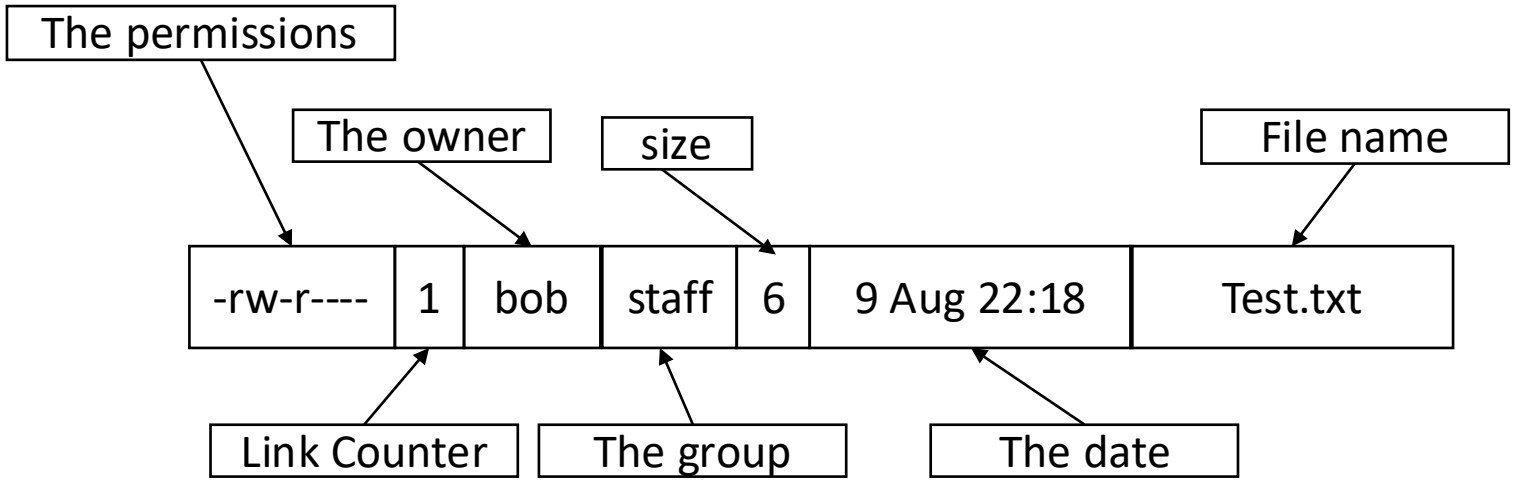
- modern UNIX systems support ACLs: FreeBSD, OpenBSD, Linux, Solaris
- FreeBSD
  - Setfacl command assigns a list of UNIX user IDs and groups
  - any number of users and groups can be associated with a file
  - read, write, execute protection bits
  - a file does not need to have an ACL
  - includes an additional protection bit that indicates whether the file has an extended ACL
- when a process requests access to a file system object two
- steps are performed:
  - step 1 selects the most appropriate ACL: owner, named users, owning / named groups, others
  - step 2 checks if the matching entry contains sufficient permissions

Access Control in Unix/Linux

- Unix/Linux/Mac use ACL, with groups.
- “uid” set when you log on.
- Linux Kernel then dynamically enforces the ACLS.
- ls –l displays files with their ACL
- `root` owns everything (“get root” = control the system)

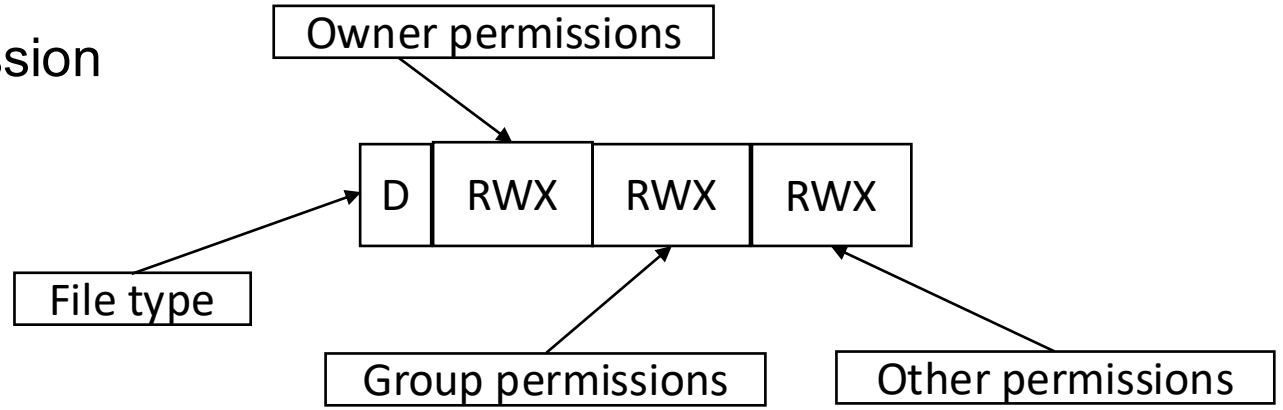


The UNIX Access Control List



UNIX File Permissions

- Permissions:
  - r: read permission
  - w: write permission
  - x: execution permission
  - -: no permissions
- File Type:
  - - : file
  - d : directory
  - b/c: device file



Mandatory Access Control (MAC)

So the *mandatory* part is that it is mandated by an authority and individual files/principles can't circumvent that.

- Systems administrator sets the policy as to who can access what
- Typical implementations of you might see:
  - SELinux: NSA's patches for Linux to add a MAC. Complex but powerful.
  - AppArmour: Canonical's simplified MAC system based on paths
  - Sandboxd which is Apple's
- Policies can be applied to more than just files

  For example: "*The web browser can access files in the downloads folder ONLY*"
- Rules are enforced on every attempted access, not at the discretion of any system user;

This is used to enforce security policies at an organizational level… and clearances.

e.g. a SECRET user may not read TOPSECRET files

SELinux basics

You define *types* that get added to *files and processes* by the policy…

You define *contexts and domains* in which the labels matter… (via a *lot* of macros)

The label grants whether an action is permitted when a given context is active…

```
allow httpd_t mysqld_etc_t:dir {
  getattr search open read lock ioctl
};
```



Role-Based Access Control

- Role-based access control (RBAC) is a widely used security framework claimed to be especially appropriate for commercial settings.
- Unlike access control policies that assign permissions to subjects, RBAC associates permissions with functions/jobs/roles within an organization.
- A role is a collection of job functions. Roles within a bank might include: president, manager, trainer, teller, auditor, janitor, etc.

Role-Based Access Control

Associate permissions with job functions

- Each job defines a set of tasks
- The tasks need permissions
- The permissions define a role

- Bank Teller:
  - Read/Write to client accounts
  - Cannot create new accounts
  - Cannot create a loan
- Role defines only the permissions allowed for the job

Role-Based Access Control

An individual has:

- a set of authorized roles, which it is allowed to fill at various times;
- a set of active roles, which it currently occupies.

Roles have an associated set of transactions, which are the activities that someone in that role is permitted to carry out.

- The set of transactions can be organization specific: open an account, cash a check, transfer funds, etc.

Role-Based Access Control

- The following are the three primary RBAC rules:
  - Role assignment: A subject can execute a transaction only if the subject has an active role.
  - Role authorization: A subject's active role must be an authorized role for that subject.
  - Transaction authorization: A subject can execute a transaction only if the transaction is authorized for one of the subject's active roles.
- Note that a subject can have multiple roles. For example, a bank president might also act as a teller.

Attribute-based Access Control

- In the attribute-based access control mechanism, access rights are defined upon attributes.
- The users' identities are not used anymore to define access rights.
- In ABAC, to get access, users need to prove their possession of the required attributes. Thus, users are authenticated to the server to prove their rights.
- ABAC is an extension of RBAC where users' roles are more generalized.
- User access rights are expressed using a set of attributes.

Sandbox

Sandboxing is a cybersecurity procedure in which you run code, analyze it, and code in a secure, enclosed environment on a system that resembles end-user working environments. It is intended to prevent the potential threat from entering the network and is commonly used to scrutinize unknown or non-secure code.

Running Untrusted Code

We often need to run buggy/untrusted code:

–programs from untrusted Internet sites: toolbars, viewers, codecs for media player

–old or insecure applications

–legacy software

=> Goal: if application “misbehaves,” kill it.

Kill it is, perhaps, an oversimplification…
Depending on what we’re sandboxing we *might* want to just log, deny or return dummy data…

Examples

Web Pages: Your browser essentially sandboxes the web pages it loads. Web pages can run JavaScript code, but this code can not do anything it wants — if JavaScript code tries to access a local file on your computer, the request will fail.

PDFs: Adobe Reader now runs PDF files in a sandbox, preventing them from escaping the PDF viewer and tampering with the rest of your computer.

In general, any application that runs in a VM (i.e. a browser or interpreter) is *to a degree* sandboxed… but that sandbox tends to be fairly permissive by default

Examples

Mobile Apps: Mobile platforms run their apps in a sandbox. Apps for iOS, Android, and Windows 8 are restricted from doing many of the things standard desktop applications can do. The sandbox disconnects the applications, stopping them from interfering with one another.

Windows Programs: User Account Control functions as a bit of a sandbox, essentially restricting Windows desktop applications from modifying system files without first asking you permission. User Account Control just restricts access to system files and system- wide settings.

Benefits of Sandboxing

- **Prevents zero-day attacks:**

Zero-day threats exploit unknown security flaws. They are highly hazardous because manufacturers cannot issue patches unless they fully comprehend the exploited vulnerability.

Sandboxing software performs admirably in isolating these menaces. Though there is no assurance that it will prevent zero-day manipulations, sandboxing most often works for damage limitation by distancing the potential danger from the remainder of the network.

- **Enhances other security programs**

Sandboxing is an excellent complement to specific other security software, ranging from action monitoring programs to antivirus programs. Conversely, sandboxing software shields against spyware strains that your virus protection may struggle to detect.

Benefits of Sandboxing

- **Helps examine presumably malicious programs for threats**

If you're dealing with new vendors or untrustworthy software sources, you can try out a new application for threats before incorporating it.

- **Allows rigorous software tests before they are released**

Software changes should be thoroughly tested before they are released to the public. Sandboxing can be used to test new code for possible risks before it officially launches. Therefore, it has a vital role to play in application security.

And in more general keeps the principle of least privilege by ensuring that things cannot touch things that do not concern them.

Earliest example is chroot(2) and it was intended as a mechanism to keep the runtime of an application local to it more than a security tool

How To Sandbox Any Program

- **Virtual Machines:** A virtual machine program like VirtualBox or VMware creates virtual hardware devices that it uses to run an operating system.

This entire operating system is essentially sandboxed, as it does not have access to anything outside of the virtual machine.

- **Using other tools:** Sandboxie, Bufferzone

Seccomp

https://www.usenix.org/conference/soups2025/presentation/alhindi

- Linux's sandboxing mechanism is called seccomp…
- It is a weird assembly language implemented atop a programming language for firewall rules (BPF)
- You define arbitrary rules for *system calls* and can programmatically inspect arguments

- No one can use it comfortably.

Playing in the Sandbox: A Study on the Usability of Seccomp

Maysara Alhindi
University of Bristol

Joseph Hallett
University of Bristol

Abstract

Sandboxing restricts what applications do, and prevents exploited processes being abused; yet relatively few applications get sandboxed: why? We report a usability trial with 7 experienced Seccomp developers exploring how they approached sandboxing an application and the difficulties they faced. The developers each approached sandboxing the application differently and each came to different solutions. We highlight many challenges of using Seccomp, the sandboxing designs by the participants, and what developers think would make it easier for them to sandbox applications effectively.

1 Introduction

on the number of sandboxed packages available in different OSs found that only 802 packages for the popular Debian and Fedora Linux OSs directly used sandboxing in their code out of the tens of thousands of packages available [2].

One reason for the relatively low sandbox adoption is that Seccomp (the Linux sandboxing framework) is *too* powerful—it provides a general programming language for sandboxing with an assembly-esque syntax that can be used by developers in many different ways and can be difficult to understand and employ correctly—even for developers who have used Seccomp before. Developers have to make design decisions when using Seccomp: they must decide what they want to sandbox in a program, with what permissions, and why. By examining how existing developers use Seccomp, we can

No seriously…

We did this study looking at what got Sandboxed in various Linuxen and BSDs…

Barely any of the Linuxen sandbox things *by default*…

And when they do… they tend to try and copy the default rules of OpenBSD where *everything* is Sandboxed by default…

why?

Table 5: Processes that make use of sandboxing features immediately on boot following a clean install.

OS	Post-boot processes	Sandboxed
Debian	120	5 (4.16%)
Fedora	137	16 (11.67%)
OpenBSD	36	33 (91.66%)
FreeBSD	47	3 (6.38%)

Sandboxing Adoption in Open Source Ecosystems

Maysara Alhindi
University of Bristol
United Kingdom
maysara.alhindi@bristol.ac.uk

Joseph Hallett
University of Bristol
United Kingdom
joseph.hallett@bristol.ac.uk

ABSTRACT

Sandboxing mechanisms allow developers to limit how much access applications have to resources, following the least-privilege principle. However, it's not clear how much and in what ways developers are using these mechanisms. This study looks at the use of *Seccomp*, *Landlock*, *Capsicum*, *Pledge*, and *Unveil* in all packages of four open-source operating systems. We found that less than 1% of packages directly use these mechanisms, but many more indirectly use them. Examining how developers apply these mechanisms reveals interesting usage patterns, such as cases where developers simplify their sandbox implementation. It also highlights challenges that may be hindering the widespread adoption of sandboxing mechanisms.

ACM Reference Format:
Maysara Alhindi and Joseph Hallett. 2024. Sandboxing Adoption in Open Source Ecosystems. In *2024 12th ACM/IEEE International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems (SESoS '24), April 14, 2024, Lisbon, Portugal.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3643655.3643878

Table 1: Different OSs examined and their packages.

OS	Version	Packages
Debian Linux	Bullseye 11	59,913
Fedora Linux	37	66,166
OpenBSD	7.3	7,787
FreeBSD	13.2	30,766

the OS, disabling unsupported features, aligning installation paths, and applying patches for OS-specific functionality.

Various operating systems offer different sandboxing mechanisms. Linux employs *Seccomp* and the Linux Security Module (LSM) framework, allowing for pluggable kernel modules like *Landlock*. FreeBSD has *Capsicum* for sandboxing, while OpenBSD en-

pledge(2) and unveil(2)

OpenBSD has two sandboxing mechanisms…

- pledge reduces the systemcalls you can make to a predefined list
  - for example, net for networking, libc for libc
- unveil says whether you can see a file…
  - everything else is hidden once you pledge to use it

There is no way to disable it once enabled… just 2 systemcalls

EVERYTHING uses it

(OpenBSD can make everything use it because the OS and its runtime update together and are controlled… not built from disparate components like Linux is… so they do the work or core and send changes upstream for packages… and because its two easily auditable lines…)

Reading references

[1] Stallings, W., Cryptography and Network Security: Principles and Practice, 7th ed., 2017, Pearson.

[2] Goodrich M. and Tamassia Roberto, Introduction to computer security, Boston, MA : Pearson, 2011.

[3]- Bishop M., Introduction to computer security, Chapter 12, *first printing,* 2004.

[4] Prevelakis, V. and Spinellis, D., 2001, June. Sandboxing Applications. In USENIX Annual Technical Conference, FREENIX Track (pp. 119-126).

[5] Greamo, C. and Ghosh, A., 2011. Sandboxing and virtualization: Modern tools for combating malware. IEEE Security & Privacy, 9(2), pp.79-82.